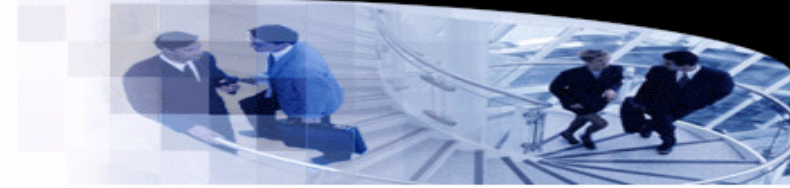




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

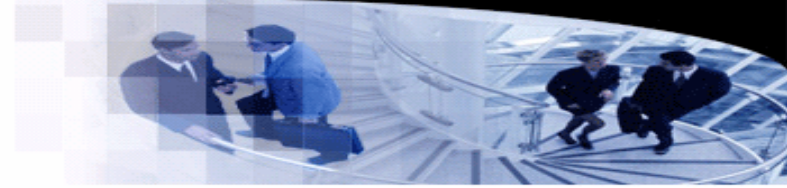
Lecture 15. LR(k) Parsing

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

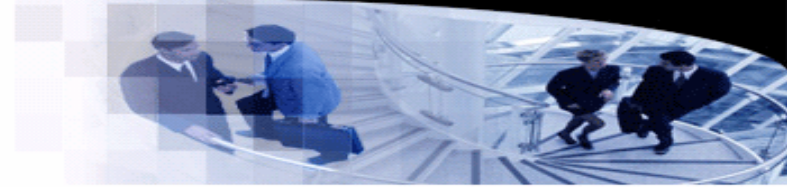
Bottom-up Parsing



- Shift-reduce parser

- Given an input α , top-down parsers would try to find a derivation such that $S \Rightarrow^* \alpha$. Finding such derivation can be depicted as constructing a syntax tree from the root to leaf nodes.
- Naturally, there are other parsing algorithms that constructs the syntax tree from leaves (**bottom**) to the root (**up**).
- Compared to top-down parsers who perform derivation actions, bottom-up parsers perform **reductions**, which are the invert actions of derivations.
- Given a production $A \rightarrow \gamma$, the reduction action replaces γ with A .

Bottom-up Parsing



- Shift-reduce parser

- Idea:

- The parser uses a stack to store intermediate strings.
 - **Reduce**: Whenever there is a “reducible” string (the right-hand side string of some production) in the stack, the parser replaces the reducible string with the left-hand side nonterminal.
 - **Shift**: If there is no string to reduce, the parser moves the current input symbol to the stack, so the stack may contain a new reducible string in the future.

- An implementation with pushdown automata

- A new start symbol S' and a new production $S' \rightarrow S$. (The parser should stop once S' is in the stack.)
 - The automaton chooses a production **nondeterministically** and tries to perform a reduction on the stack.
 - The automaton shift the current input symbol to the stack **nondeterministically**.
 - When S' is on the top of the stack, the parser pops S' . Now if the stack and the input are both empty, the parser accept the input.

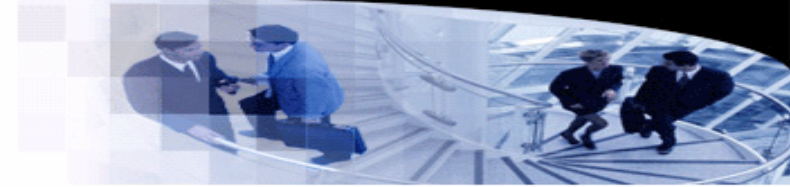
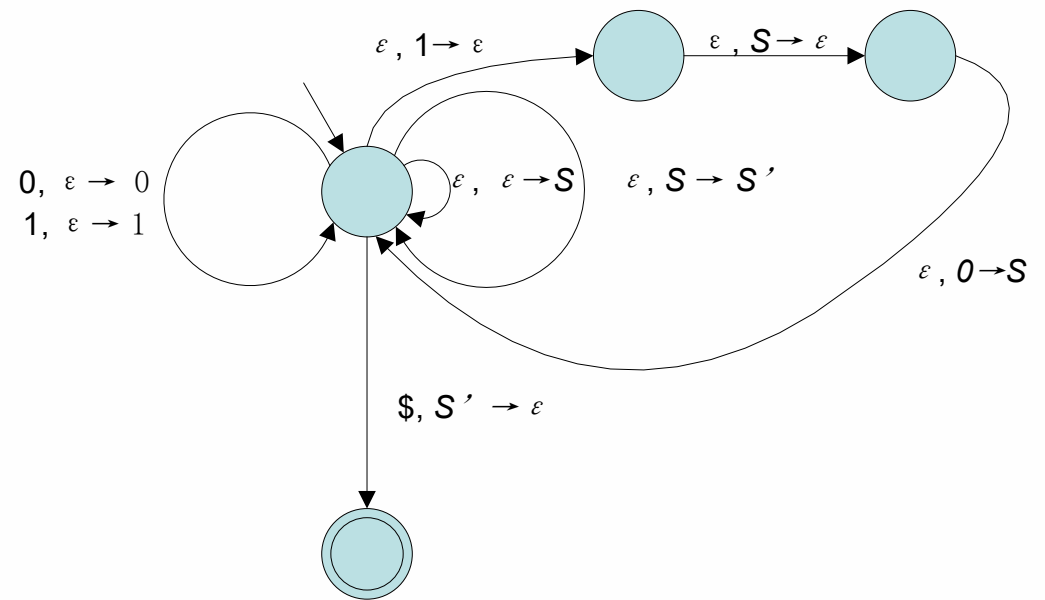
Bottom-up Parsing

- Shift-reduce parser

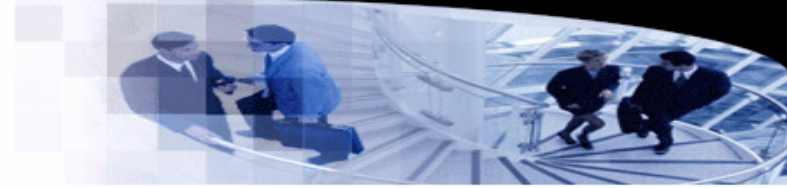
- Consider a grammar $G[S']$ as follows.

- $S' \rightarrow S$
- $S \rightarrow 0S1 \mid \epsilon$

- The shift-reduce automaton is presented as the right-hand side.
- Note that the automaton is used to describe the idea of the parser.
- It has some flaws. For example, this automaton accepts $001\$$.



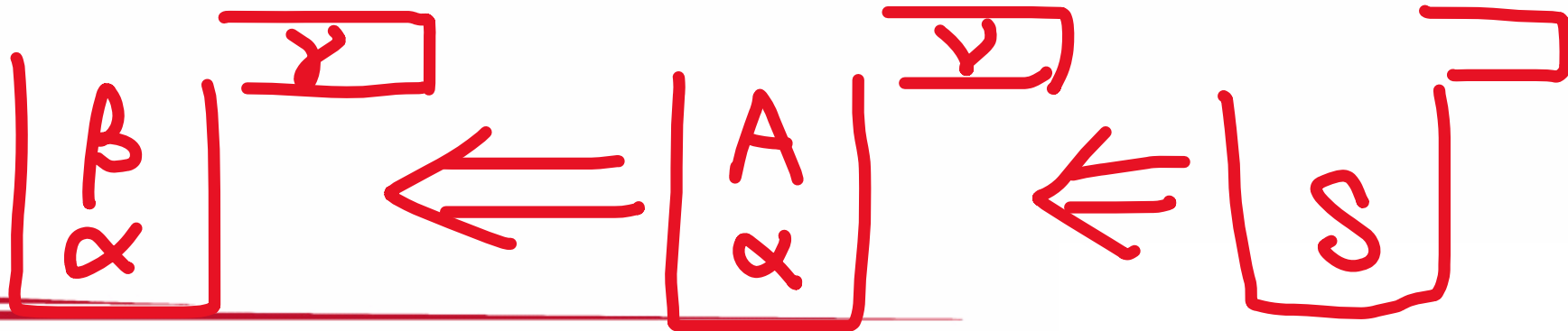
Bottom-up Parsing



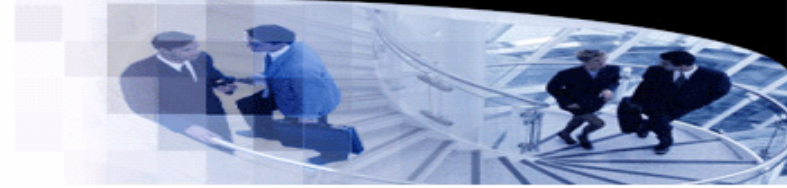
- Shift-reduce parser

- The nondeterministic choices, again, is the key to the performance issue.
- We need a mechanism to check whether there is a “reducible” string on the top of the stack.
- A “reducible” string is formally denoted as a **handle**.
- Note that not every right-hand side of a production is reducible.
 - For example, there is always an ϵ on the top of the stack. The parser only chooses to reduce ϵ in certain contexts (e.g., with a “0” on the top of the stack and an “1” being the current input).
- **Definition:** If $A \rightarrow \beta$ and $S \Rightarrow^* \alpha A \gamma \Rightarrow \alpha \beta \gamma$ where $\gamma \in V_T^*$, then β is called the handle of $\alpha \beta \gamma$.

why is β
“reducible” in
this case.



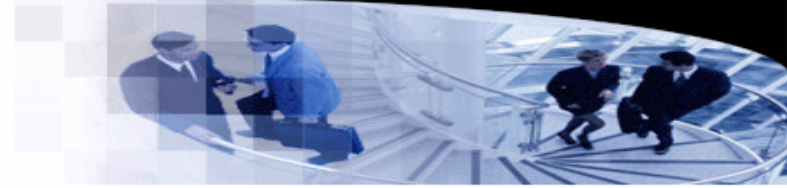
Bottom-up Parsing



- Shift-reduce parser

- Given a well-formed input, if the parser performs the reduction action whenever a handle appears on the top of the stack, then the input will be correctly reduced to the start symbol.
- Handles, unfortunately, can only be confirmed after the whole syntax tree is constructed.
- Hence, it is impossible to find handles during parsing for an arbitrary grammar.
- The theory of automata tells us that the nondeterminism is inevitable for pushdown automata to recognize context-free grammars.
- For the purpose of parser design, we only consider grammars that are recognized by deterministic pushdown automata.
- In some of these grammars, handles can be identified efficiently.

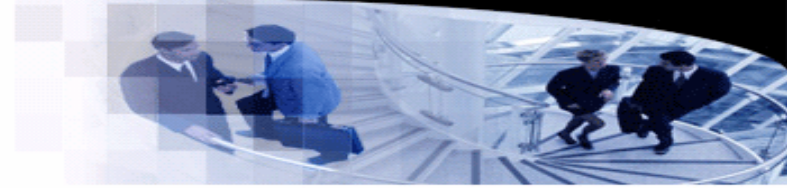
LR(0) Parsing



- LR(0) grammar

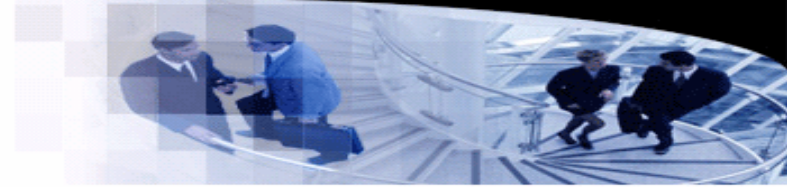
- One of the LR family of grammars.
- Handles in LR(0) grammars can be recognized by a heuristic algorithm, computing **viable prefixes**.
- **Definition:** In a shift-reduce parser, a string α in the stack is called a viable prefix, if there is a β such that $S \Rightarrow^* \alpha\beta$.
- α is a viable prefix means that
 - no error has been detected by the parser,
 - and it is a prefix of a handle.
- In LR(0) grammars, if $A \rightarrow \gamma$ and $\beta\gamma$ is a viable prefix, then γ is a handle.
- The set of all viable prefixes of an LR(0) grammar is a regular language, and therefore can be recognized by an finite automaton.

LR(0) Parsing



- Finite automaton to recognize viable prefixes
 - States:
 - A state is called an LR(0) item with the form $A \rightarrow \alpha \cdot \beta$.
 - $A \rightarrow \alpha \cdot \beta$ represents the progress of recognizing a handle $\alpha\beta$ in a viable prefix, where α is on the top of the stack and β is expected to be put into the stack.
 - $A \rightarrow \alpha \cdot$ means that the handle α is on the top of the stack, and therefore should be reduced.
 - Transitions:
 - Given an item $A \rightarrow \alpha \cdot X\beta$, there is a transition that accepts X and goes to $A \rightarrow \alpha X \cdot \beta$.
 - Given an item $A \rightarrow \alpha \cdot B\beta$ and $B \rightarrow \gamma_1|\gamma_2| \dots |\gamma_n$, there are ϵ -transitions going to every $B \rightarrow \cdot \gamma_i$.

LR(0) Parsing



- Finite automaton to recognize viable prefixes

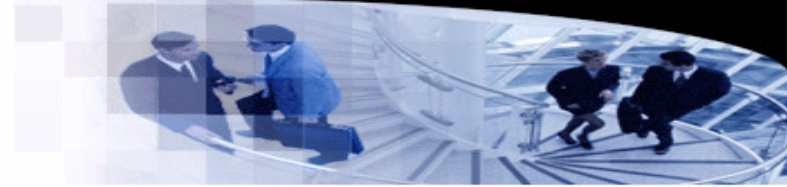
- States:

- A state is called an LR(0) item with the form $A \rightarrow \alpha \cdot \beta$.
- $A \rightarrow \alpha \cdot \beta$ represents the progress of recognizing a handle $\alpha\beta$ in a viable prefix, where α is on the top of the stack and β is expected to be put into the stack.
- $A \rightarrow \alpha \cdot$ means that the handle α is on the top of the stack, and therefore should be reduced.

- Transitions:

- Given an item $A \rightarrow \alpha \cdot X\beta$, there is a transition that accepts X and goes to $A \rightarrow \alpha X \cdot \beta$.
- Given an item $A \rightarrow \alpha \cdot B\beta$ and $B \rightarrow \gamma_1 | \gamma_2 | \dots | \gamma_n$, there are ϵ -transitions going to every $B \rightarrow \cdot \gamma_i$.

LR(0) Parsing



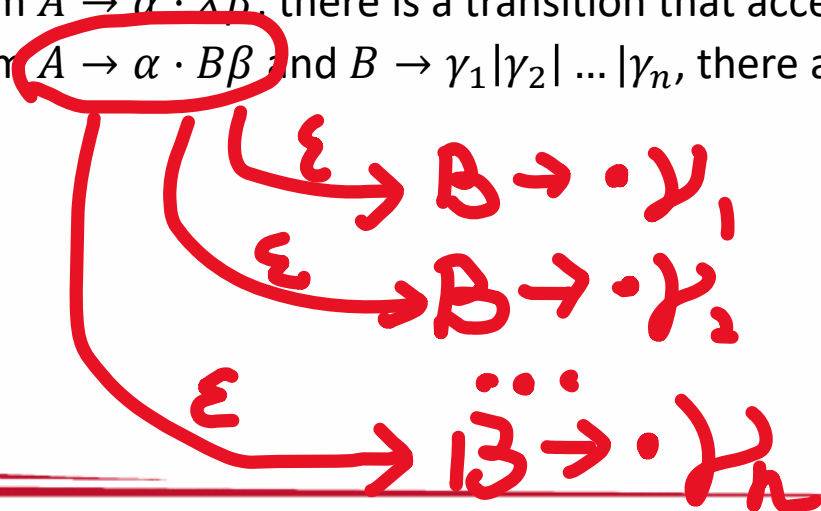
- Finite automaton to recognize viable prefixes

- States:

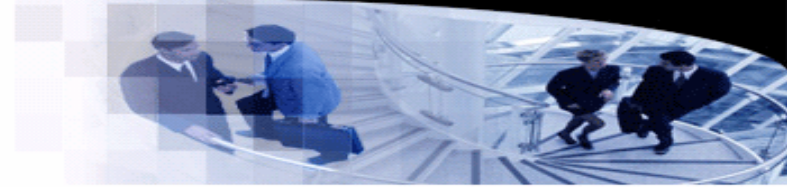
- A state is called an LR(0) item with the form $A \rightarrow \alpha \cdot \beta$.
- $A \rightarrow \alpha \cdot \beta$ represents the progress of recognizing a handle $\alpha\beta$ in a viable prefix, where α is on the top of the stack and β is expected to be put into the stack.
- $A \rightarrow \alpha \cdot$ means that the handle α is on the top of the stack, and therefore should be reduced.

- Transitions:

- Given an item $A \rightarrow \alpha \cdot X\beta$, there is a transition that accepts X and goes to $A \rightarrow \alpha X \cdot \beta$.
- Given an item $A \rightarrow \alpha \cdot B\beta$ and $B \rightarrow \gamma_1 | \gamma_2 | \dots | \gamma_n$, there are ϵ -transitions going to every $B \rightarrow \cdot \gamma_i$.



LR(0) Parsing



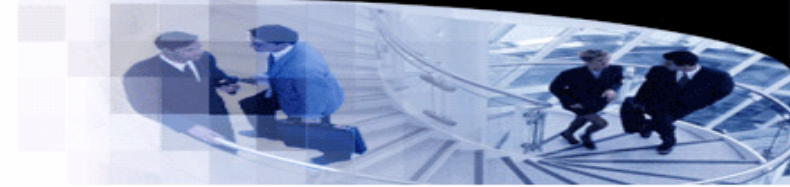
- Finite automaton to recognize viable prefixes
 - The intuition of the ϵ -transition going from $A \rightarrow \alpha \cdot B\beta$ to $B \rightarrow \gamma_i$ is that:
 - We hope B will soon be pushed into the stack to advance the recognition progress.
 - However, we also welcome other strings that can be reduced to B , which can be pushed into the stack subsequently.
 - The only problem is that, when γ_i is reduced to B , how to let the automaton go back to $A \rightarrow \alpha \cdot B\beta$, so as to accept B on the stack?
 - The solution is to setup a state stack that is **synchronous** to the symbol stack, which was the original stack.
 - The **top state** on the state stack indicates the current state of the automaton.

LR(0) Parsing

- Finite automaton to recognize viable prefixes
 - We demonstrate how this mechanism works.

	γ	β	...
--	----------	---------	-----

α	$A \rightarrow \alpha \cdot B\beta$
...	...

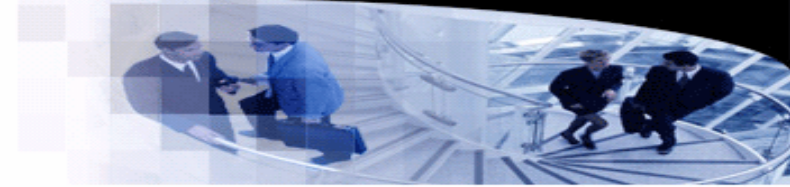


LR(0) Parsing

- Finite automaton to recognize viable prefixes
 - We demonstrate how this mechanism works.
 - Note that this is an NFA.

	γ	β	...
--	----------	---------	-----

α	$A \rightarrow \alpha \cdot B\beta \quad B \rightarrow \cdot \gamma$
...	...

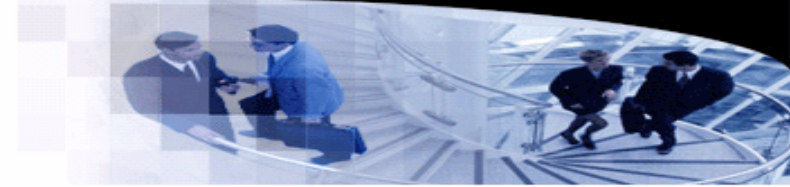


LR(0) Parsing

- Finite automaton to recognize viable prefixes
 - We demonstrate how this mechanism works.
 - Note that this is an NFA.

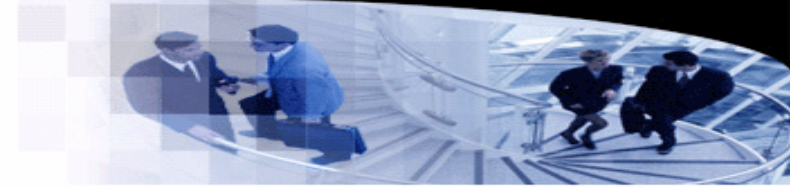
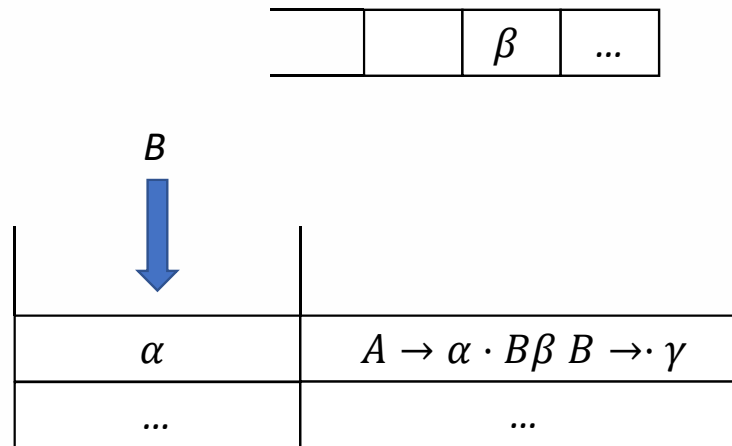
		β	...
--	--	---------	-----

γ	$B \rightarrow \gamma \cdot$
α	$A \rightarrow \alpha \cdot B\beta \quad B \rightarrow \cdot \gamma$
...	...

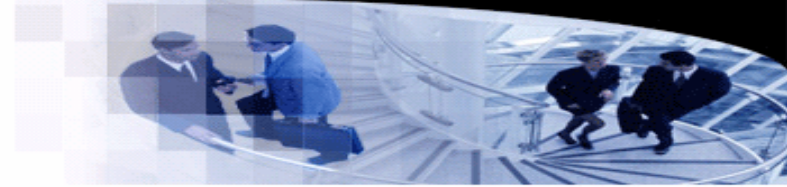


LR(0) Parsing

- Finite automaton to recognize viable prefixes
 - We demonstrate how this mechanism works.
 - Bingo! The automaton may accept B now.



LR(0) Parsing

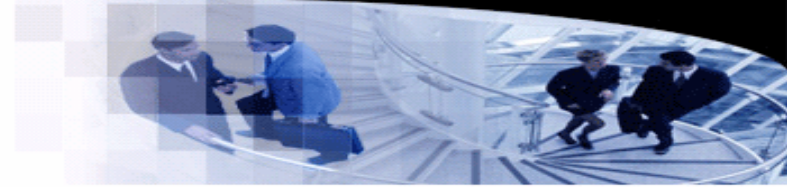


- Finite automaton to recognize viable prefixes
 - We demonstrate how this mechanism works.
 - The progress of recognizing $A \rightarrow \alpha B \beta$ has been advanced.

		β	...
--	--	---------	-----

B	$A \rightarrow \alpha B \cdot \beta$
α	$A \rightarrow \alpha \cdot B \beta \quad B \rightarrow \cdot \gamma$
...	...

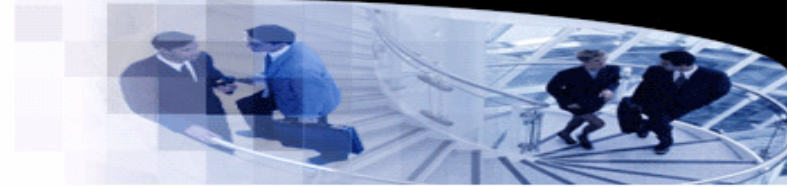
LR(0) Parsing



- Finite automaton to recognize viable prefixes
 - The deterministic version of the automaton can be constructed by the following steps.
 - 1. Construct the initial item $S' \rightarrow \cdot S$, and let $\epsilon - Closure(\{S' \rightarrow \cdot S\})$ be the initial state.
 - $\epsilon - Closure(X) = \{q \mid q \text{ is reachable by only } \epsilon\text{-transitions from some } p \in X.\}$
 - 2. Construct a new state X according to transitions from states that have already been constructed. Add $\epsilon - Closure(X)$ to the set of states.
 - 3. Repeat 2 until no new states are constructed.

LR(0) Parsing

- Finite automaton to recognize viable prefixes
 - Example grammar:
 - 1. $S' \rightarrow E$
 - 2. $E \rightarrow aB$
 - 3. $E \rightarrow bA$
 - 4. $A \rightarrow cA$
 - 5. $A \rightarrow d$
 - 6. $B \rightarrow cB$
 - 7. $B \rightarrow d$

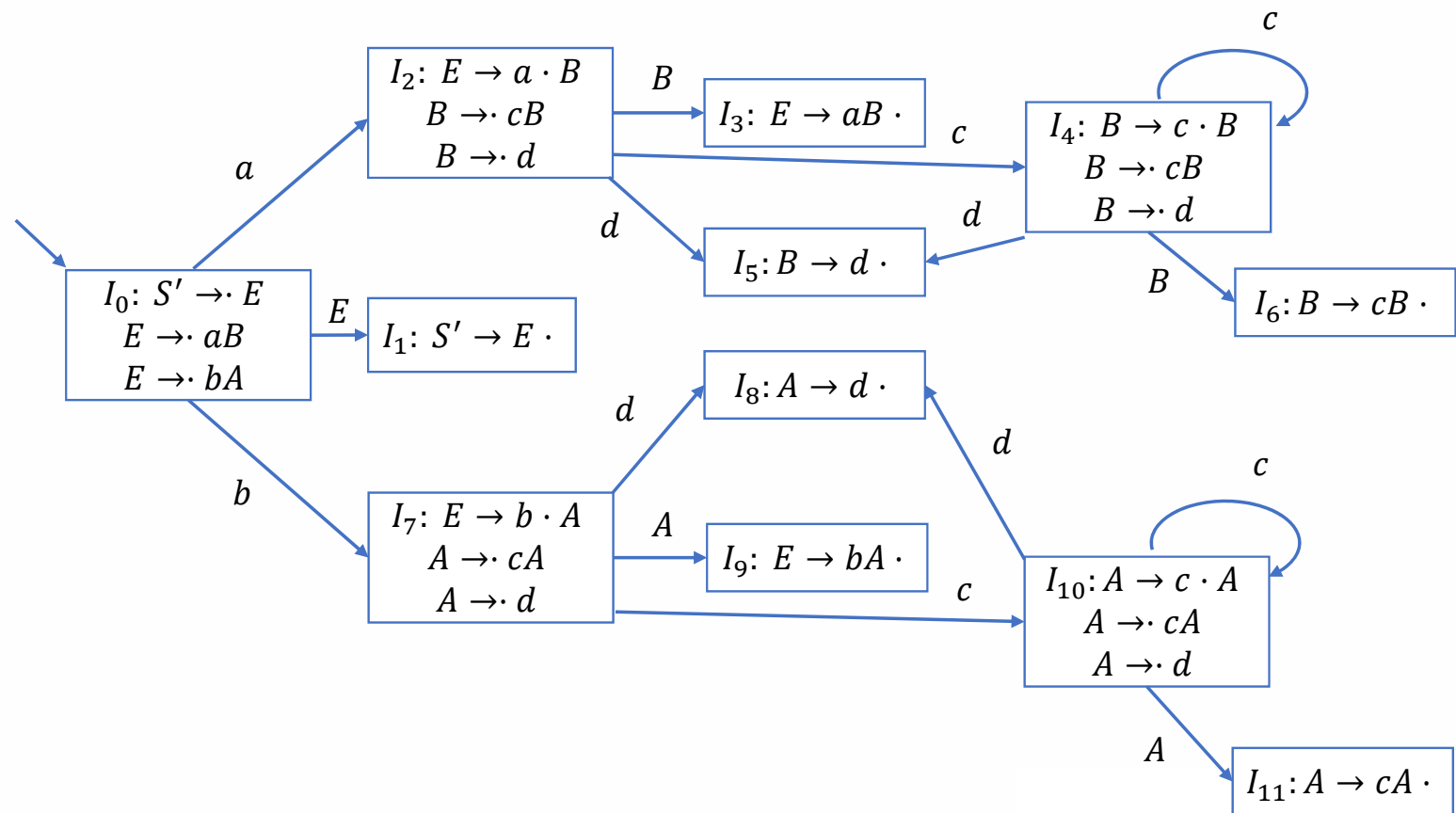


LR(0) Parsing

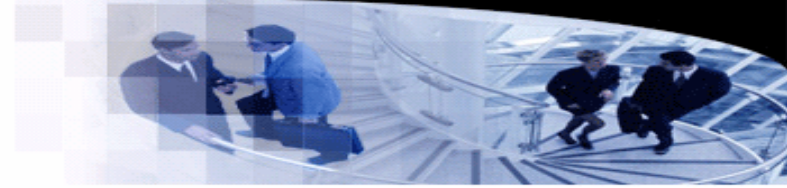
- Finite automaton to recognize viable prefixes

- Example:

- 1. $S' \rightarrow E$
- 2. $E \rightarrow aB$
- 3. $E \rightarrow bA$
- 4. $A \rightarrow cA$
- 5. $A \rightarrow d$
- 6. $B \rightarrow cB$
- 7. $B \rightarrow d$



LR(0) Parsing



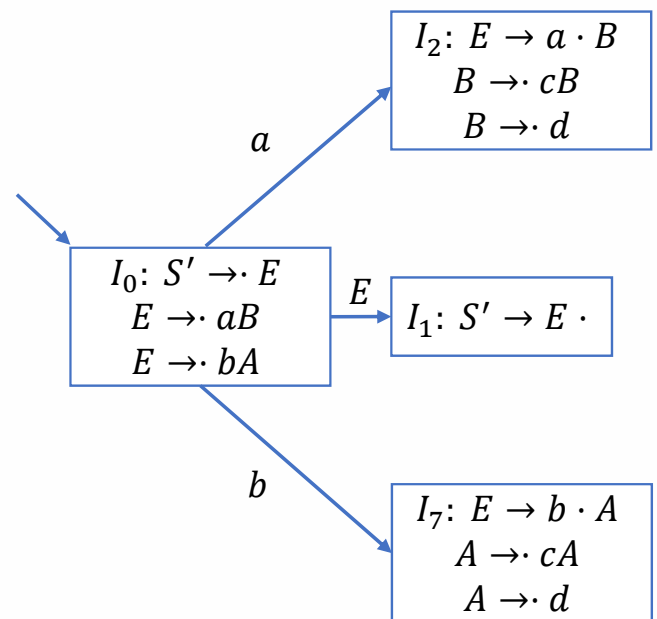
- Parsing table

- The LR(0) parsing table is a function mapping the current state and the input symbol to a **parsing action**.
- A parsing action can be a shift action or a reduce action.
 - A shift action s_i indicates the parser to push the current symbol to the symbol stack and let i be the state on the top of the state stack.
 - A reduce action r_j indicates the parser to perform a reduction by using the j^{th} production.
- The action is determined by the LR(0) items in the state.
 - If there is an item $A \rightarrow \gamma \cdot$ in the state, then it is a reduction action for any input symbol.
 - If there is an item $A \rightarrow \beta \cdot a\gamma$ in the state, then it is a shift action for the input a .
- Conflicts:
 - If there exists a reduction and a shift action on a state, it is called a **shift-reduce** conflict.
 - If there are more than one reduction action on a state, it is called a **reduce-reduce** conflict.
- Grammars with conflicts are **NOT** LR(0) grammar.

LR(0) Parsing

- Parsing table
- Example

	a	b	c	d	\$	A	B	E	S'
0	s_2	s_7						s_1	
1									
2									
3									
4									
5									
6									
7									
8									
9									
10									
11									



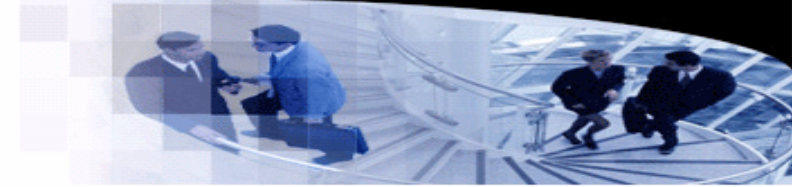
LR(0) Parsing

- Parsing table
- Example

	a	b	c	d	\$	A	B	E	S'
0	s_2	s_7						s_1	
1	r_1	r_1	r_1	r_1	acc				
2									
3									
4									
5									
6									
7									
8									
9									
10									
11									

$E \rightarrow I_1: S' \rightarrow E \cdot$

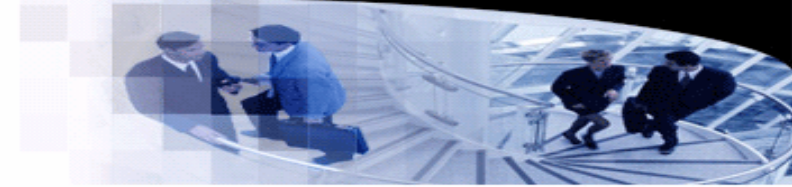
1. $S' \rightarrow E$
2. $E \rightarrow aB$
3. $E \rightarrow bA$
4. $A \rightarrow cA$
5. $A \rightarrow d$
6. $B \rightarrow cB$
7. $B \rightarrow d$



LR(0) Parsing

- Parsing table
 - Example

	a	b	c	d	\$	A	B	E	S'
0	s_2	s_7						s_1	
1	r_1	r_1	r_1	r_1	acc				
2			s_4	s_5			s_3		
3	r_2	r_2	r_2	r_2	r_2				
4			s_4	s_5			s_6		
5	r_7	r_7	r_7	r_7	r_7				
6	r_6	r_6	r_6	r_6	r_6				
7			s_{10}	s_8		s_9			
8	r_5	r_5	r_5	r_5	r_5				
9	r_3	r_3	r_3	r_3	r_3				
10			s_{10}	s_8		s_{11}			
11	r_4	r_4	r_4	r_4	r_4				



LR(0) Parsing

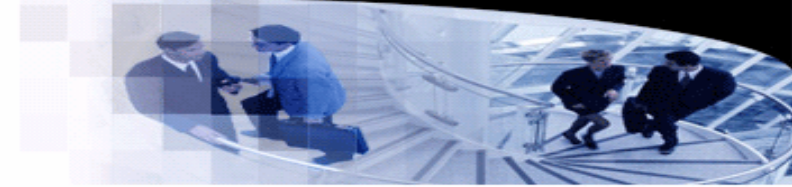
• Parsing table

• Example

- 1. $S' \rightarrow E$
- 2. $E \rightarrow aB$
- 3. $E \rightarrow bA$
- 4. $A \rightarrow cA$
- 5. $A \rightarrow d$
- 6. $B \rightarrow cB$
- 7. $B \rightarrow d$

	a	b	c	d	\$	A	B	E	S'
0	s_2	s_7						s_1	
1	r_1	r_1	r_1	r_1	acc				
2			s_4	s_5			s_3		
3	r_2	r_2	r_2	r_2	r_2				
4			s_4	s_5			s_6		
5	r_7	r_7	r_7	r_7	r_7				
6	r_6	r_6	r_6	r_6	r_6				
7			s_{10}	s_8		s_9			
8	r_5	r_5	r_5	r_5	r_5				
9	r_3	r_3	r_3	r_3	r_3				
10			s_{10}	s_8		s_{11}			
11	r_4	r_4	r_4	r_4	r_4				

Symol Stack	State Stack	Input	Action
\$	0	accd\$	s_2
a\$	2 0	ccd\$	s_4
ca\$	4 2 0	cd\$	s_4
cca\$	4 4 2 0	d\$	s_5
dcca\$	5 4 4 2 0	\$	r_7
cca\$	4 4 2 0	\$	Push B using s_6 .
Bcca\$	6 4 4 2 0	\$	r_6
ca\$	4 2 0	\$	Push B using s_6 .
Bca\$	6 4 2 0	\$	r_6
a\$	2 0	\$	Push B using s_3 .



LR(0) Parsing

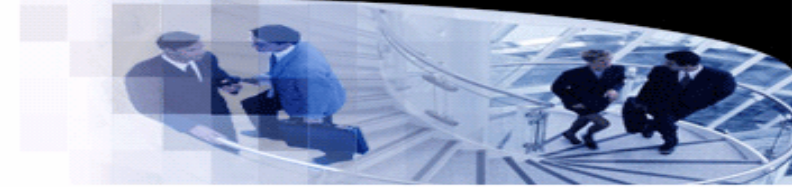
• Parsing table

• Example

- 1. $S' \rightarrow E$
- 2. $E \rightarrow aB$
- 3. $E \rightarrow bA$
- 4. $A \rightarrow cA$
- 5. $A \rightarrow d$
- 6. $B \rightarrow cB$
- 7. $B \rightarrow d$

	a	b	c	d	\$	A	B	E	S'
0	s_2	s_7						s_1	
1	r_1	r_1	r_1	r_1	acc				
2			s_4	s_5			s_3		
3	r_2	r_2	r_2	r_2	r_2				
4			s_4	s_5			s_6		
5	r_7	r_7	r_7	r_7	r_7				
6	r_6	r_6	r_6	r_6	r_6				
7			s_{10}	s_8		s_9			
8	r_5	r_5	r_5	r_5	r_5				
9	r_3	r_3	r_3	r_3	r_3				
10			s_{10}	s_8		s_{11}			
11	r_4	r_4	r_4	r_4	r_4				

Symol Stack	State Stack	Input	Action
a\$	2 0	\$	Push B using s_3 .
Ba\$	3 2 0	\$	r_2
\$	0	\$	Push E using s_1 .
E\$	1 0	\$	r_1
\$	0	\$	Push S'. Accept.



SLR(1) Parsing

- Conflict handling

- Consider the following grammar and the LR(0) automaton.

- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow (E) \mid i$

- A shift-reduce conflict happens for '+'.
 - The stack contains "(", "x", "+".

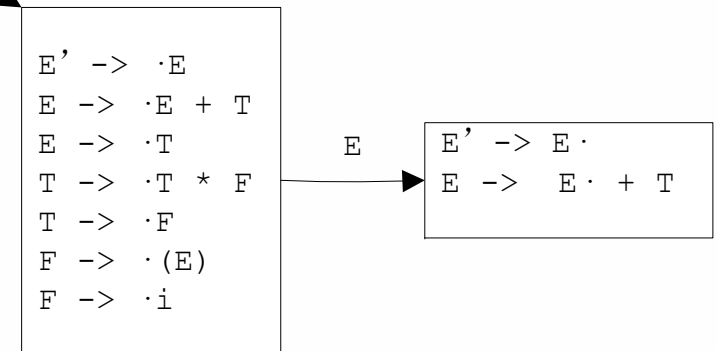
- $E' \rightarrow E \cdot$ hopes to reduce E to E' .
- $E \rightarrow E \cdot +T$ expects to shift a '+'.
 (if not, error)

- Observation: E cannot be reduced if the input is '+', as the only symbol that may occur following E' is '\$'.

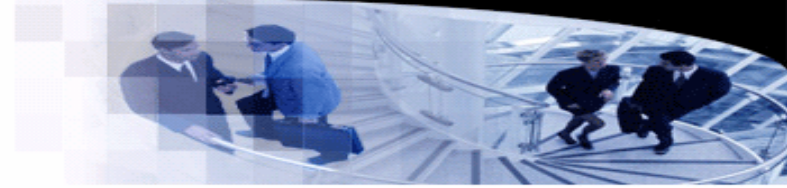
- $+ \notin \text{Follow}(E')$ in another word.

- SLR(1) parsing

- $A \rightarrow \alpha \cdot$ is reducible only if the input symbol $a \in Follow(A)$.



LR(k) Parsing



- Other LR(k) grammars

- LR(1) items are of the form $A \rightarrow \alpha \cdot \beta, a$, which indicates that $\alpha\beta$ is a handle only if the look ahead symbol is a .
- LR(1) algorithms can resolve much more conflicts than that of SLR(1).
- The disadvantage of LR(1) is the state explosion caused by the look ahead symbol.
- LALR(1) parsers reduce state by merging similar states.
 - For example, $A \rightarrow \alpha \cdot \beta, a$ and $A \rightarrow \alpha \cdot \beta, b$ are merged into a single state $A \rightarrow \alpha \cdot \beta, a/b$.

